

BÁO CÁO THỰC HÀNH: KỸ NĂNG VIẾT PROMPT HIỆU QUẢ (PROMPT ENGINEERING)

I. Mục tiêu Báo cáo này trình bày kết quả thử nghiệm và phân tích kỹ năng viết prompt nhằm tận dụng tối đa khả năng của các Mô hình Ngôn ngữ Lớn (LLMs) trong môi trường học tập và nghiên cứu kỹ thuật. Quá trình thực nghiệm được tiến hành qua 3 tác vụ phổ biến, đối chiếu kết quả giữa các cấp độ prompt: Cơ bản, Cải tiến và Nâng cao.

II. Nội dung thực hiện và So sánh kết quả(Ảnh minh chứng ở những trang cuối)

Tác vụ học tập	Phiên bản Prompt	Đặc điểm của Kết quả đầu ra (Output)	Đánh giá s Phân tích nguyên nhân
1. Tóm tắt tài liệu học thuật (Soft Robotics)	Cơ bản	Một đoạn văn bản liền mạch, thông tin dàn trải, văn phong chung chung.	Yếu: Thiếu định dạng và ngữ cảnh khiến AI không biết đâu là trọng tâm, người đọc khó nắm bắt ý chính.
	Cải tiến	Trình bày theo gạch đầu dòng, bám sát được 3 ý chính yêu cầu (Nguyên lý, ưu điểm, ứng dụng).	Khá: Kỹ thuật <i>Task Breakdown</i> giúp kết quả có cấu trúc và dễ theo dõi hơn.
	Nâng cao	Văn bản chuẩn học thuật kỹ thuật, cấu trúc vi mô mạch lạc với tiêu đề in đậm, ngôn từ chuyên ngành chính xác.	Xuất sắc: <i>Role-prompting</i> kích hoạt từ vựng chuyên ngành. <i>Structural Constraints</i> ép AI lấp đầy thông tin đúng trọng tâm, loại bỏ hoàn toàn sự lan man.

Tác vụ học tập	Phiên bản Prompt	Đặc điểm của Kết quả đầu ra (Output)	Đánh giá & Phân tích nguyên nhân
2. Giải thích khái niệm phức tạp (Lập trình Python)	Cơ bản	Chỉ đưa ra định nghĩa lý thuyết mang tính sách vở, không có tính ứng dụng.	Yếu: Câu lệnh quá mở, thiếu bối cảnh thực hành khiến AI trả lời như một cuốn từ điển.
	Cải tiến	Đưa ra định nghĩa, có so sánh cơ bản và kèm theo một đoạn code minh họa.	Khá: Việc yêu cầu rõ "có so sánh và ví dụ code" giúp tăng tính trực quan của câu trả lời.
	Nâng cao	Giải thích sâu về hiệu suất, ví dụ code có comment giải thích từng dòng, có lời khuyên áp dụng thực tế.	Xuất sắc: Kỹ thuật <i>Chain-of-Thought</i> buộc AI phân tích lý do sâu xa trước khi code. <i>Role-prompting</i> (Senior Dev) đảm bảo code viết ra tối ưu và có comment chuẩn mực.
3. Tạo bộ câu hỏi ôn tập (Môn Triết học)	Cơ bản	Các câu hỏi ngẫu nhiên, chủ yếu thiên về học thuộc lòng khái niệm.	Yếu: Thiếu định hướng về độ khó và mục tiêu đánh giá năng lực.
	Cải tiến	Cấu trúc rõ ràng gồm 5 câu trắc nghiệm, 2 câu tự luận và có kèm đáp án.	Khá: Ràng buộc được số lượng và thể loại câu hỏi, hỗ trợ việc ôn tập cơ bản tốt.

Tác vụ học tập	Phiên bản Prompt	Đặc điểm của Kết quả đầu ra (Output)	Đánh giá s Phân tích nguyên nhân
	Nâng cao	Câu hỏi trắc nghiệm mang tính phản biện, câu tự luận là tình huống thực tế, kèm theo <i>thang điểm</i> chấm chi tiết.	Xuất sắc: Kỹ thuật <i>Few-shot Prompting</i> định chuẩn ngay lập tức độ khó. Yêu cầu tạo <i>thang điểm</i> biến prompt thành một công cụ kiểm tra đánh giá toàn diện.

3. Phân tích hiệu quả: Điểm khác biệt tinh tế nhất ở Prompt Nâng cao nằm ở kỹ thuật **Few-shot Prompting**. Bằng cách cung cấp sẵn một câu hỏi tự luận mẫu, người viết đã định chuẩn ngay lập tức độ khó và định hướng tư duy (hỏi về ứng dụng thực tế thay vì học thuộc lòng định nghĩa) cho AI. Thêm vào đó, việc bổ sung yêu cầu về "Thang điểm chấm (Rubric)" biến AI từ một công cụ soạn văn bản thành một hệ thống hỗ trợ tự đánh giá năng lực học tập toàn diện.

III. Tổng hợp các nguyên tắc và mẹo viết Prompt hiệu quả

Dựa trên các kết quả đối chiếu thực nghiệm, để tối ưu hóa hiệu năng của LLMs, quá trình Prompt Engineering cần tuân thủ 4 nguyên tắc hệ thống sau:

- Nguyên tắc Role-Prompting:** Việc thiết lập một nhân dạng chuyên môn (ví dụ: Kỹ sư, Lập trình viên Senior, Giảng viên) giúp kích hoạt chính xác cơ sở dữ liệu chuyên ngành của AI, tăng tính học thuật và độ tin cậy của câu trả lời.
- Nguyên tắc Formatting Constraints:** Cần quy định rõ định dạng đầu ra. Sự ràng buộc khắt khe này giúp kiểm soát luồng thông tin, giảm thiểu sự lan man không cần thiết.
- Nguyên tắc Chain-of-Thought:** Với các tác vụ phân tích, việc yêu cầu AI tư duy theo từng bước hoặc theo một sườn logic cho trước giúp hệ thống xử lý vấn đề sâu sắc hơn, giảm tỷ lệ lỗi logic.

"Hãy tóm tắt tài liệu về Soft Robotics. Yêu cầu trình bày theo dạng gạch đầu dòng và tập trung bám sát 3 ý chính sau: 1. Nguyên lý hoạt động, 2. Ưu điểm, 3. Ứng dụng thực tế."

Tóm tắt Soft Robotics (Robot mềm)

1. Nguyên lý hoạt động

- Soft Robotics là công nghệ chế tạo robot từ các vật liệu mềm, đàn hồi như silicone, cao su, hydrogel hoặc polymer thông minh.
- Thay vì sử dụng khung kim loại và khớp cứng, robot mềm thay đổi hình dạng bằng cách uốn cong, co giãn hoặc xoắn.
- Chuyển động được tạo ra thông qua:
 - **Khí nén (Pneumatic Actuation):** bơm khí vào các khoang mềm để tạo chuyển động.
 - **Thủy lực (Hydraulic Actuation):** sử dụng chất lỏng áp suất cao để tạo lực.
 - **Vật liệu thông minh:** thay đổi hình dạng khi chịu tác động của nhiệt, điện trường hoặc từ trường.
- Thiết kế thường lấy cảm hứng từ các sinh vật như bạch tuộc, giun đất hoặc cơ bắp sinh học của con người.

2. Ưu điểm

- **An toàn khi tương tác với con người** do cấu trúc mềm, giảm nguy cơ va chạm gây chấn thương.
- **Linh hoạt và thích nghi cao**, có thể hoạt động trong môi trường phức tạp hoặc không xác định.
- **Khả năng thao tác vật thể dễ vỡ** như trái cây, thực phẩm, trứng mà không làm hỏng chúng.
- **Trọng lượng nhẹ**, dễ mang theo và tích hợp vào các thiết bị đeo hỗ trợ con người.
- **Khả năng mô phỏng chuyển động sinh học**, giúp robot vận động tự nhiên hơn so với robot cứng truyền thống.

3. Ứng dụng thực tế

- **Y tế**
 - Robot hỗ trợ phẫu thuật ít xâm lấn.
 - Thiết bị phục hồi chức năng và hỗ trợ vận động.
 - Ống nội soi và dụng cụ y sinh mềm.
- **Công nghiệp**
 - Tay gắp mềm trong dây chuyền sản xuất.
 - Thu hoạch và phân loại nông sản.
 - Đóng gói thực phẩm và sản phẩm dễ vỡ.
- **Khám phá và cứu hộ**
 - Di chuyển trong không gian hẹp, đường ống hoặc khu vực đổ nát.
 - Khảo sát môi trường dưới nước và địa hình phức tạp.
- **Thiết bị đeo (Wearable Robotics)**
 - Bộ hỗ trợ vận động cho người cao tuổi.
 - Exosuit mềm hỗ trợ công nhân và bệnh nhân phục hồi chức năng.

Kết luận

- Soft Robotics là hướng phát triển robot sử dụng vật liệu mềm để tạo ra các hệ thống linh hoạt, an toàn và thích nghi cao.
- Công nghệ này có tiềm năng lớn trong y tế, công nghiệp, cứu hộ và các thiết bị hỗ trợ con người nhờ khả năng tương tác hiệu quả với môi trường và con người.

"Đóng vai là một chuyên gia nghiên cứu Robot học, hãy tóm tắt các kiến thức về Soft Robotics. Yêu cầu cấu trúc khắt khe: Sử dụng các tiêu đề in đậm cho từng phần. Chỉ sử dụng ngôn từ chuyên ngành kỹ thuật chính xác. Trình bày đúng trọng tâm vào Nguyên lý, Ưu điểm và Ứng dụng, tuyệt đối không viết lan man hay dùng từ ngữ chung chung."

SOFT ROBOTICS (ROBOT MỀM)

1. NGUYÊN LÝ HOẠT ĐỘNG

- Soft Robotics là lĩnh vực robot học sử dụng vật liệu có mô đun đàn hồi thấp (silicone elastomer, polyurethane, hydrogel, vật liệu composite mềm) để tạo cấu trúc có khả năng biến dạng liên tục (continuous deformation).
- Cơ chế chuyển động không dựa trên chuỗi khớp cứng (rigid-link mechanism) mà dựa trên sự biến dạng đàn hồi của vật liệu dưới tác động của cơ cấu chấp hành (actuator).
- Các phương pháp truyền động chủ yếu:
 - Pneumatic Actuation:** sử dụng áp suất khí nén để tạo giãn nở không đồng đều trong các khoang khí, sinh ra chuyển động uốn, xoắn hoặc co rút.
 - Hydraulic Actuation:** sử dụng chất lỏng áp suất cao để tạo lực lớn và độ cứng hiệu dụng cao hơn hệ khí nén.
 - Electroactive Polymer (EAP):** vật liệu biến dạng dưới tác động của điện trường.
 - Shape Memory Materials:** vật liệu nhớ hình thay đổi cấu trúc hình học khi chịu kích thích nhiệt hoặc điện.
 - Magnetically Responsive Materials:** vật liệu đàn hồi tích hợp hạt từ tính, điều khiển bằng từ trường ngoài.
- Mô hình động học và động lực học của robot mềm thường được xây dựng dựa trên:
 - Lý thuyết biến dạng liên tục (Continuum Mechanics).
 - Phân tích phần tử hữu hạn (Finite Element Analysis - FEA).
 - Mô hình Cosserat Rod.
 - Điều khiển phi tuyến và điều khiển thích nghi.

2. ƯU ĐIỂM

- Độ tuân thủ cơ học cao (High Compliance):**
 - Hấp thụ và chạm hiệu quả.
 - Giảm ứng suất tiếp xúc với môi trường và con người.
- An toàn trong tương tác người - robot (Human-Robot Interaction):**
 - Giảm nguy cơ gây chấn thương so với robot công nghiệp khung cứng.
- Khả năng thích nghi hình học (Morphological Adaptation):**
 - Tự động ôm sát hình dạng vật thể mà không cần điều khiển phức tạp.
- Bậc tự do hiệu dụng lớn (High Effective Degrees of Freedom):**
 - Tạo chuyển động liên tục trong không gian làm việc.
 - Dễ tiếp cận các vùng hạn chế về hình học.
- Khả năng thao tác vật thể dễ hư hỏng:**
 - Phân bố áp lực tiếp xúc đồng đều.
 - Hạn chế biến dạng hoặc phá hủy vật thể.
- Khả năng làm việc trong môi trường phi cấu trúc (Unstructured Environment):**
 - Hoạt động hiệu quả trong không gian chật hẹp, địa hình phức tạp hoặc môi trường sinh học.

3. ỨNG DỤNG THỰC TẾ

Y tế và kỹ thuật y sinh

- Robot phẫu thuật mềm cho can thiệp ít xâm lấn (Minimally Invasive Surgery).
- Catheter và endoscope mềm điều hướng trong hệ mạch hoặc cơ quan nội tạng.
- Thiết bị hỗ trợ phục hồi chức năng và exosuit mềm.
- Tay giả sinh học có độ tuân thủ cơ học cao.

Tự động hóa công nghiệp

- Soft Gripper trong dây chuyền đóng gói.
- Gắp và phân loại thực phẩm, trái cây, linh kiện dễ biến dạng.
- Hệ thống thao tác cộng tác (Collaborative Manipulation) với con người.

Nông nghiệp thông minh

- Thu hoạch nông sản có bề mặt mềm hoặc dễ dập.

Nông nghiệp thông minh

- Thu hoạch nông sản có bề mặt mềm hoặc dễ dập.
- Phân loại sản phẩm nông nghiệp bằng tay gấp mềm thích nghi hình dạng.

Khám phá và kiểm tra kỹ thuật

- Robot mềm khảo sát đường ống, khoang hẹp và hệ thống ngầm.
- Robot dưới nước mô phỏng sinh vật biển để giám sát môi trường.
- Hệ thống kiểm tra không phá hủy (Non-Destructive Inspection) trong các kết cấu phức tạp.

Cứu hộ và ứng phó thảm họa

- Robot có khả năng lườn qua khe hẹp trong đồng đồ nát.
- Thực hiện nhiệm vụ tìm kiếm và thu thập dữ liệu tại khu vực nguy hiểm hoặc khó tiếp cận.

KẾT LUẬN

Soft Robotics là hướng phát triển robot dựa trên cấu trúc biến dạng liên tục và vật liệu đàn hồi, trong đó cơ chế vận động được tạo bởi các cơ cấu chấp hành mềm thay vì hệ khớp cứng truyền thống. Giá trị cốt lõi của công nghệ nằm ở độ tuân thủ cơ học cao, khả năng thích nghi hình học và mức độ an toàn vượt trội khi tương tác với môi trường hoặc con người, tạo lợi thế rõ rệt trong các ứng dụng y sinh, thao tác vật thể mềm, robot cộng tác và hệ thống làm việc trong môi trường phi cấu trúc.

TÁC VỤ 2:

Python

Định nghĩa khái niệm Decorator trong lập trình Python là gì?

Decorator trong Python là gì?

Decorator là một cơ chế trong Python cho phép mở rộng hoặc thay đổi hành vi của hàm (function) hoặc lớp (class) mà không cần sửa đổi mã nguồn gốc của đối tượng đó.

Về bản chất, decorator là một hàm nhận một hàm khác làm đối số và trả về một hàm mới với chức năng được bổ sung.

Nguyên lý hoạt động

Python coi hàm là đối tượng hạng nhất (first-class object), nghĩa là:

- Hàm có thể được gán cho biến.
- Hàm có thể được truyền làm tham số.
- Hàm có thể được trả về từ một hàm khác.

Decorator tận dụng đặc điểm này để “bao bọc” (wrap) hàm gốc bằng một hàm mới.

Ví dụ cơ bản

Python

```
def decorator(func):
    def wrapper():
        print("Thực hiện trước hàm gốc")
        func()
        print("Thực hiện sau hàm gốc")
    return wrapper

@decorator
def hello():
    print("Hello Python!")

hello()
```

Python

Kết quả:

Thực hiện trước hàm gốc

Hello Python!

Thực hiện sau hàm gốc

Cú pháp:

Python

```
@decorator
def hello():
    pass
```

tương đương với:

Python

```
hello = decorator(hello)
```

Mục đích sử dụng

Decorator thường được dùng để bổ sung các chức năng chung mà không làm thay đổi logic chính của chương trình:

- Logging (ghi log)
- Kiểm tra quyền truy cập
- Xác thực người dùng
- Đo thời gian thực thi
- Caching (lưu kết quả tạm thời)
- Xử lý ngoại lệ
- Theo dõi hiệu năng

Ví dụ: Đo thời gian thực thi

 Python



 Run

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()

        result = func(*args, **kwargs)

        end = time.time()
        print(f"Execution time: {end - start:.4f}s")

        return result
    return wrapper

@timer
def calculate():
    time.sleep(2)

calculate()
```

Kết quả:

Execution time: 2.0001s



Decorator có tham số

Decorator cũng có thể nhận tham số riêng.

 Python



 Run

```
def repeat(n):
    def decorator(func):
        def wrapper():
            for _ in range(n):
```



Python



Run

```
def wrapper():
    for _ in range(n):
        func()
    return wrapper
return decorator

@repeat(3)
def greet():
    print("Hello")

greet()
```

Kết quả:

Hello
Hello
Hello



Decorator có sẵn trong Python

Một số decorator được sử dụng phổ biến:

Decorator	Chức năng
<code>@staticmethod</code>	Khai báo phương thức tĩnh trong class
<code>@classmethod</code>	Phương thức làm việc với class thay vì instance
<code>@property</code>	Biến phương thức thành thuộc tính
<code>@functools.lru_cache</code>	Cache kết quả hàm
<code>@dataclass</code>	Tự động sinh constructor và các phương thức cho lớp dữ liệu

Ví dụ:



Ví dụ:

Python



Run

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def area(self):
        return 3.14 * self.radius ** 2

c = Circle(5)
print(c.area)
```

Ở đây `area` được truy cập như một thuộc tính thay vì gọi hàm.

"Hãy giải thích khái niệm Decorator trong Python. Hãy đưa ra so sánh cơ bản để dễ hiểu và cung cấp một đoạn code minh họa cách sử dụng."



Decorator trong Python là gì?

Decorator là một cơ chế cho phép bạn **bổ sung chức năng** cho một hàm hoặc phương thức mà không cần sửa đổi mã nguồn của hàm đó.

Nói cách khác, decorator hoạt động như một "lớp bao bọc" (wrapper) xung quanh hàm gốc. Khi hàm được gọi, decorator có thể thực hiện thêm một số tác vụ trước hoặc sau khi hàm gốc chạy.

So sánh để dễ hiểu

Hãy tưởng tượng một **món quà**:

- Hàm gốc = món quà.
- Decorator = giấy gói quà.

Giấy gói không làm thay đổi món quà bên trong, nhưng nó bổ sung thêm tính năng hoặc giá trị bên ngoài.

Tương tự:

```
</> Python
send_email()
```



là chức năng gốc.

Bạn có thể dùng decorator để:

- Ghi log trước khi gửi email.
- Kiểm tra quyền người dùng.
- Đo thời gian thực thi.
- Xử lý lỗi.

mà không cần chỉnh sửa mã bên trong `send_email()`.



Cách hoạt động

Decorator là một hàm nhận vào một hàm khác và trả về một hàm mới.

Sơ đồ đơn giản:

```
Hàm gốc
↓
Decorator
↓
Hàm mới đã được bổ sung chức năng
```



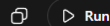
Cú pháp:

```
</> Python
@decorator_name
def my_function():
    pass
```



tương đương với:

```
</> Python
my_function = decorator_name(my_function)
```



Ví dụ minh họa

Bước 1: Tạo decorator

```
</> Python
def log_decorator(func):

    def wrapper():
        print("Bắt đầu thực thi hàm...")

        func()
```



</> Python



▶ Run

```
func()

print("Kết thúc thực thi hàm...")

return wrapper
```

Bước 2: Áp dụng decorator

</> Python



▶ Run

```
@log_decorator
def hello():
    print("Xin chào Python!")
```

Bước 3: Gọi hàm

</> Python



▶ Run

```
hello()
```

Kết quả

```
Bắt đầu thực thi hàm...
Xin chào Python!
Kết thúc thực thi hàm...
```



Ví dụ thực tế hơn: Đo thời gian thực thi

</> Python



▶ Run

```
import time

def timer(func):
```



```

</> Python
import time

def timer(func):

    def wrapper(*args, **kwargs):
        start = time.time()

        result = func(*args, **kwargs)

        end = time.time()

        print(f"Thời gian chạy: {end - start:.3f} giây")

        return result

    return wrapper

@timer
def process_data():
    time.sleep(2)
    print("Đang xử lý dữ liệu...")

process_data()

```

Kết quả:

```

Đang xử lý dữ liệu...
Thời gian chạy: 2.001 giây

```

Ở đây:

- `process_data()` chứa nghiệp vụ chính.
- Decorator `@timer` tự động đo thời gian thực thi.
- Nếu cần đo thời gian cho nhiều hàm khác, chỉ cần thêm `@timer` mà không phải viết lại mã đo thời gian.

"Bạn là một Lập trình viên Senior Python. Hãy giải thích khái niệm Decorator. Hãy tư duy theo các bước sau: Bước 1: Phân tích lý do sâu xa tại sao cần dùng Decorator và giải thích sâu về hiệu suất. Bước 2: Viết một ví dụ code minh họa tối ưu, yêu cầu có comment giải thích chi tiết cho từng dòng code. Bước 3: Đưa ra lời khuyên thực tế về việc áp dụng kỹ thuật này trong các dự án lớn."

Bước 1: Tại sao cần Decorator? Góc nhìn của Senior Python Developer

Vấn đề thực tế trong dự án lớn

Khi hệ thống phát triển, bạn sẽ gặp rất nhiều chức năng "phụ trợ" xuất hiện lặp đi lặp lại:

- Logging
- Authentication / Authorization
- Cache
- Retry khi gọi API
- Transaction Database
- Performance Monitoring
- Exception Handling
- Rate Limiting

Ví dụ:

```

</> Python
def get_user(user_id):
    logger.info("Start")

    result = query_user(user_id)

    logger.info("End")
    return result

def get_order(order_id):
    logger.info("Start")

```

 Python



 Run

```
result = query_order(order_id)

logger.info("End")
return result
```

Sau vài tháng:

500 hàm
1000 hàm
5000 hàm



Bạn sẽ thấy code business bị "ô nhiễm" bởi các đoạn xử lý phụ trợ.

Lúc này xuất hiện khái niệm:

Cross-Cutting Concerns

Tức là các chức năng xuyên suốt toàn hệ thống nhưng không thuộc nghiệp vụ chính.

Decorator sinh ra để giải quyết chính xác vấn đề này.

Decorator giúp gì?

Thay vì:

 Python



 Run

```
def process_payment():
    log_start()
    check_permission()

    # Business Logic

    log_end()
```

Ta viết:



 Python



 Run

```
@logger
@permission_required
def process_payment():
    # Business Logic
```

Business Logic được tách biệt hoàn toàn khỏi Infrastructure Logic.

Đây là nguyên tắc rất quan trọng:

Separation of Concerns (SoC)



Decorator và hiệu suất (Performance)

Đây là điểm nhiều lập trình viên hiểu chưa đầy đủ.

Decorator KHÔNG làm code chạy nhanh hơn

Ví dụ:

 Python



 Run

```
@logger
def calculate():
    pass
```

Decorator thực chất thêm một lớp gọi hàm:

```
calculate()
↓
wrapper()
↓
calculate_original()
```



Do đó:

- Có thêm 1 function call



Do đó:

- Có thêm 1 function call
- Có thêm stack frame

=> Chi phí nhỏ nhưng vẫn tồn tại.

Về mặt CPU:

Decorator thuần túy
=
Slight Overhead



Khi Decorator giúp tăng hiệu suất

Decorator trở nên cực kỳ mạnh khi được dùng cho:

Caching

Ví dụ:

```
</> Python
from functools import lru_cache
```



▶ Run

Một phép tính Fibonacci:

```
</> Python
fib(40)
```



▶ Run

Nếu không cache:

$O(2^n)$



Nếu cache:

$O(n)$



Tốc độ có thể nhanh hơn hàng nghìn lần.

Đây là ví dụ kinh điển mà decorator thực sự cải thiện hiệu suất hệ thống.

Decorator trong các framework lớn

Các framework Python sử dụng decorator rất nhiều:

- [Flask](#)

```
</> Python
@app.route("/users")
```



Run

- [FastAPI](#)

```
</> Python
@app.get("/users")
```



Run

- [Django](#)

```
</> Python
@login_required
```



Run

Bản chất đều là:

```
</> Python
function
↓
decorator
↓
new function
```



Run

Bước 2: Ví dụ tối ưu với comment chi tiết

Ví dụ thực tế: Logging + Đo thời gian thực thi.

 Python



 Run

```
# Import module đo thời gian
import time

# Import wraps để giữ metadata của hàm gốc
from functools import wraps

# Decorator dùng để ghi log và đo thời gian chạy
def performance_logger(func):

    # wraps giúp giữ nguyên:
    # - __name__
    # - __doc__
    # - annotations
    @wraps(func)
    def wrapper(*args, **kwargs):

        # Lưu thời điểm bắt đầu
        start_time = time.perf_counter()

        print(f"[START] Running: {func.__name__}")

        try:
            # Thực thi hàm gốc
            result = func(*args, **kwargs)

            # Trả về kết quả cho caller
            return result

        finally:

            # Lấy thời điểm kết thúc
            end_time = time.perf_counter()

            # Tính thời gian thực thi
            execution_time = end_time - start_time
```



 Python



 Run

```
        print(
            f"[END] {func.__name__} "
            f"executed in {execution_time:.6f} seconds"
        )

    # Trả về hàm wrapper
    return wrapper

# Áp dụng decorator
@performance_logger
def process_data(n):
    """
    Mô phỏng xử lý dữ liệu lớn
    """

    total = 0

    for i in range(n):
        total += i

    return total

# Gọi hàm
result = process_data(10_000_000)

print(result)
```

Luồng hoạt động thực tế

Khi Python đọc:

 Python



 Run

```
@performance_logger
def process_data():
```



</> Python



▶ Run

Python tự động chuyển thành:

</> Python



▶ Run

```
process_data = performance_logger(process_data)
```

Sau đó:

</> Python



▶ Run

```
process_data()
```

thực tế là:

</> Python



▶ Run

```
wrapper()
```

và bên trong wrapper:

</> Python



▶ Run

```
func()
```

chính là hàm gốc.

Luồng gọi:

```
process_data()
↓
wrapper()
↓
log_start
↓
process_data_original()
↓
log_end
```



```
log_end
↓
return result
```

Bước 3: Lời khuyên thực tế trong dự án lớn

1. Chỉ dùng Decorator cho Cross-Cutting Concerns

Tốt:

```
</> Python
@cache
@logger
@retry
@transaction
```



▶ Run

Không tốt:

```
</> Python
@calculate_salary
@calculate_tax
@calculate_bonus
```



▶ Run

Nhiệm vụ chính không nên bị nhét vào decorator.

Decorator nên phục vụ hạ tầng hệ thống.

2. Luôn dùng @wraps

Sai:

```
</> Python
def decorator(func):
    def wrapper():
```



▶ Run

Python



Run

```
def wrapper():  
    return func()  
return wrapper
```

Kết quả:

Python



Run

```
print(my_func.__name__)  
  
# wrapper
```

Thông tin hàm gốc bị mất.

Đúng:

Python



Run

```
from functools import wraps  
  
@wraps(func)
```

Đây gần như là quy tắc bắt buộc trong code production.

3. Tránh xếp quá nhiều Decorator

Ví dụ:

Python



Run

```
@cache  
@retry  
@logger  
@permission  
@transaction  
@monitoring  
def service():
```



`</> Python`



`▶ Run`

```
@monitoring  
def service():
```

Vẫn chạy được.

Nhưng sau 2 năm:

Debug nightmare.



Việc truy vết lỗi trở nên khó khăn.

Nên giới hạn số lớp decorator ở mức hợp lý.

4. Cẩn thận với Decorator Stateful

Ví dụ:

`</> Python`



`▶ Run`

```
counter = 0
```

được dùng trong decorator.

Trong môi trường:

- Multi-thread
- Async
- Multi-process

rất dễ phát sinh race condition.

Nếu cần trạng thái, hãy dùng:

`</> Python`



`▶ Run`

```
threading.Lock
```

contextvars
Redis
Database

tùy tình huống.

5. Decorator mạnh nhất khi kết hợp Cache

Một trong những ứng dụng hiệu quả nhất:

 Python



 Run

```
from functools import lru_cache

@lru_cache(maxsize=1024)
def expensive_calculation(x):
    ...
```

Áp dụng cho:

- Dynamic Programming
- API lookup
- Machine Learning feature calculation
- Configuration loading
- Query dữ liệu ít thay đổi

Đây là trường hợp decorator vừa giúp code sạch hơn  và cải thiện hiệu suất đáng kể.

TÁC VỤ 3:

"Hãy tạo cho tôi một số câu hỏi để ôn tập môn Triết học."



Dưới đây là bộ câu hỏi ôn tập môn Triết học được chia theo các mức độ từ nhận biết đến vận dụng và phân tích.

Phần 1: Câu hỏi khái niệm cơ bản

1. Triết học là gì? Đối tượng nghiên cứu của triết học là gì?
2. Vấn đề cơ bản của triết học gồm những nội dung nào?
3. Chủ nghĩa duy vật là gì? Có những hình thức phát triển nào?
4. Chủ nghĩa duy tâm là gì? Phân biệt duy tâm chủ quan và duy tâm khách quan.
5. Thế nào là thế giới quan? Vai trò của thế giới quan đối với con người.
6. Phương pháp luận là gì? Mối quan hệ giữa thế giới quan và phương pháp luận.
7. Phép biện chứng là gì? Đặc điểm cơ bản của phép biện chứng duy vật.
8. Siêu hình là gì? So sánh phương pháp siêu hình và phương pháp biện chứng.

Phần 2: Câu hỏi về phép biện chứng duy vật

1. Nguyên lý về mối liên hệ phổ biến được hiểu như thế nào?
2. Nguyên lý về sự phát triển có nội dung gì?
3. Phân tích cặp phạm trù nguyên nhân và kết quả.
4. Phân biệt nội dung và hình thức.
5. Phân tích mối quan hệ giữa bản chất và hiện tượng.
6. Khả năng và hiện thực khác nhau như thế nào?
7. Quy luật chuyển hóa từ những thay đổi về lượng thành những thay đổi về chất là gì?
8. Khái niệm "độ", "điểm nút" và "bước nhảy" trong triết học.
9. Nội dung quy luật thống nhất và đấu tranh của các mặt đối lập.
10. Vì sao quy luật mâu thuẫn được xem là nguồn gốc của sự vận động và phát triển?
11. Nội dung quy luật phủ định của phủ định.
12. Tại sao sự phát triển mang tính kế thừa?

Phần 3: Câu hỏi về nhận thức luận

1. Thực tiễn là gì? Các hình thức cơ bản của thực tiễn.
 2. Vai trò của thực tiễn đối với nhận thức.
 3. Nhận thức cảm tính gồm những hình thức nào?
 4. Nhận thức lý tính gồm những hình thức nào?
 5. Mối quan hệ giữa nhận thức cảm tính và nhận thức lý tính.
 6. Chân lý là gì?
 7. Tại sao thực tiễn là tiêu chuẩn kiểm nghiệm chân lý?
 8. Phân biệt chân lý tuyệt đối và chân lý tương đối.
-

Phần 4: Câu hỏi về triết học xã hội

1. Sản xuất vật chất có vai trò như thế nào đối với sự tồn tại và phát triển của xã hội?
 2. Lực lượng sản xuất là gì? Các yếu tố cấu thành lực lượng sản xuất.
 3. Quan hệ sản xuất là gì?
 4. Mối quan hệ giữa lực lượng sản xuất và quan hệ sản xuất.
 5. Cơ sở hạ tầng là gì?
 6. Kiến trúc thượng tầng là gì?
 7. Mối quan hệ giữa cơ sở hạ tầng và kiến trúc thượng tầng.
 8. Tồn tại xã hội là gì?
 9. Ý thức xã hội là gì?
 10. Mối quan hệ giữa tồn tại xã hội và ý thức xã hội.
-

Phần 5: Câu hỏi vận dụng và phân tích

1. Hãy vận dụng quy luật lượng - chất để phân tích quá trình học tập của sinh viên.
2. Hãy lấy ví dụ thực tế chứng minh mọi sự vật đều tồn tại trong mối liên hệ phổ biến.
3. Phân tích một hiện tượng xã hội dưới góc độ nguyên nhân và kết quả.
4. Vận dụng quy luật phủ định của phủ định để giải thích sự phát triển của khoa học và công nghệ.
5. Hãy chứng minh thực tiễn là cơ sở của nhận thức bằng một ví dụ thực tế.
6. Phân tích vai trò của trí thức và công nghệ trong  phát triển lực lượng sản xuất hiện nay.
7. Liên hệ bản thân để làm rõ vai trò của thế giới quan khoa học trong học tập và công việc.

Phần 5: Câu hỏi vận dụng và phân tích

1. Hãy vận dụng quy luật lượng - chất để phân tích quá trình học tập của sinh viên.
2. Hãy lấy ví dụ thực tế chứng minh mọi sự vật đều tồn tại trong mối liên hệ phổ biến.
3. Phân tích một hiện tượng xã hội dưới góc độ nguyên nhân và kết quả.
4. Vận dụng quy luật phủ định của phủ định để giải thích sự phát triển của khoa học và công nghệ.
5. Hãy chứng minh thực tiễn là cơ sở của nhận thức bằng một ví dụ thực tế.
6. Phân tích vai trò của tri thức và công nghệ trong sự phát triển lực lượng sản xuất hiện nay.
7. Liên hệ bản thân để làm rõ vai trò của thế giới quan khoa học trong học tập và công việc.
8. Hãy chỉ ra một mâu thuẫn trong cuộc sống và phân tích cách giải quyết theo quan điểm biện chứng.

Câu hỏi thường xuất hiện trong thi tự luận

1. Phân tích nội dung và ý nghĩa phương pháp luận của quy luật lượng - chất.
2. Phân tích nội dung và ý nghĩa phương pháp luận của quy luật mâu thuẫn.
3. Phân tích vai trò của thực tiễn đối với nhận thức.
4. Phân tích mối quan hệ biện chứng giữa lực lượng sản xuất và quan hệ sản xuất.
5. Phân tích mối quan hệ giữa tồn tại xã hội và ý thức xã hội.
6. Chứng minh rằng thực tiễn là tiêu chuẩn của chân lý.
7. Phân tích nguyên lý về mối liên hệ phổ biến và ý nghĩa phương pháp luận.
8. Phân tích nguyên lý về sự phát triển và ý nghĩa phương pháp luận.

Mô tả: Few-shot Prompting định chuẩn độ khó, Rubric chấm điểm, đánh giá phản biện

Dưới đây là mô tả ngắn gọn và mang tính học thuật cho ba khái niệm:

1. Few-shot Prompting

Few-shot Prompting là kỹ thuật cung cấp cho mô hình AI một số lượng nhỏ ví dụ mẫu (thường từ 2 đến 10 ví dụ) ngay trong prompt để hướng dẫn cách thực hiện nhiệm vụ.

Mục đích

- Định hình cách suy luận của mô hình.
- Chuẩn hóa định dạng đầu ra.
- Cải thiện độ chính xác đối với các tác vụ phức tạp.

Ví dụ

Prompt:

Ví dụ 1:

Câu hỏi: $2 + 3 = ?$

Trả lời: 5

Ví dụ 2:

Câu hỏi: $7 + 8 = ?$

Trả lời: 15

Bây giờ:

Câu hỏi: $9 + 6 = ?$

Trả lời:



Mô hình sẽ học từ các ví dụ trước để tạo ra câu trả lời phù hợp.

2. Định chuẩn độ khó (Difficulty Calibration)

Định chuẩn độ khó là quá trình xác định và phân loại mức độ khó của câu hỏi hoặc bài tập theo các tiêu chí nhất quán.

Mục tiêu

- Đảm bảo đề đánh giá có tính phân hóa.
- Tránh câu hỏi quá dễ hoặc quá khó.
- Đo lường chính xác năng lực người học hoặc mô hình AI.

Ví dụ phân loại

Mức độ	Đặc điểm
Dễ	Chỉ yêu cầu ghi nhớ kiến thức
Trung bình	Yêu cầu hiểu và áp dụng
Khó	Yêu cầu phân tích và tổng hợp
Rất khó	Yêu cầu đánh giá, phản biện hoặc sáng tạo

Trong đánh giá AI

Độ khó có thể được xác định dựa trên:

- Số bước suy luận.
- Mức độ trừu tượng.
- Lượng kiến thức nền cần thiết.
- Sự mơ hồ của bài toán.

3. Rubric chấm điểm

Rubric là bảng tiêu chí đánh giá được xây dựng trước nhằm chấm điểm một cách khách quan và nhất quán.

Thành phần của Rubric

- Tiêu chí đánh giá.
- Mức độ hoàn thành.
- Điểm số tương ứng.

Ví dụ

Tiêu chí	Điểm tối đa
Tính chính xác	4
Tính đầy đủ	3
Lập luận logic	2
Trình bày rõ ràng	1
Tổng	10

Lợi ích

- Giảm tính chủ quan của người chấm.
- Dễ so sánh giữa nhiều bài làm.
- Phù hợp cho đánh giá con người và AI.

4. Đánh giá phản biện (Critical Evaluation)

Đánh giá phản biện là quá trình phân tích, kiểm tra và đánh giá chất lượng của một lập luận, quan điểm hoặc câu trả lời dựa trên bằng chứng và logic.

Các bước cơ bản

1. Xác định luận điểm chính.
2. Kiểm tra tính hợp lý của lập luận.
3. Đánh giá bằng chứng hỗ trợ.
4. Tìm điểm yếu hoặc giả định ẩn.
5. Đề xuất quan điểm hoặc giải pháp thay thế.

Ví dụ

Luận điểm:

"Học trực tuyến luôn hiệu quả hơn học trực tiếp."

Đánh giá phản biện:

- Từ "luôn" là khẳng định tuyệt đối.
- Chưa có dữ liệu chứng minh cho mọi đối tượng học tập.
- Hiệu quả còn phụ thuộc vào môn học, người học và môi trường học tập.
- Do đó, kết luận trên chưa đủ cơ sở.

Mối liên hệ giữa 4 khái niệm

Few-shot Prompting

↓

Tạo chuẩn đầu ra mong muốn

Định chuẩn độ khó

↓

Xác định mức độ thử thách của nhiệm vụ

Rubric chấm điểm

↓

Đo lường chất lượng kết quả

Đánh giá phản biện

↓

Phân tích điểm mạnh, điểm yếu và độ tin cậy của kết quả



Trong các hệ thống đánh giá AI hiện đại, bốn thành phần này thường được kết hợp với nhau để xây dựng quy trình đánh giá có tính khách quan, nhất quán và đáng tin cậy.

(Mô tả: Few-shot Prompting định chuẩn độ khó, Rubric chấm điểm, đánh giá phản biện)

"Hãy tạo một bộ câu hỏi kiểm tra môn Triết học.
Yêu cầu:

Trắc nghiệm: Tạo các câu hỏi mang tính phản biện, không hỏi định nghĩa thuần túy.

Tự luận: Tạo câu hỏi dựa trên tình huống thực tế.
Ví dụ về độ khó mong muốn của câu tự luận: 'Vận dụng quy luật lượng - chất để phân tích quá trình tự học của sinh viên trong thời đại số.' -> Hãy dựa vào mẫu này để tạo ra các câu hỏi tương đương.

Đánh giá: Phải cung cấp thang điểm chấm (Rubric) chi tiết cho phần tự luận để người học tự đánh giá năng lực."

Show less ^



Dưới đây là bộ câu hỏi được thiết kế theo hướng **đánh giá năng lực tư duy triết học**, không kiểm tra khả năng học thuộc lòng. Độ khó được định chuẩn ở mức **vận dụng và phân tích**, tương đương các dạng câu hỏi thường xuất hiện trong học phần Triết học Mác - Lênin ở bậc đại học.

PHẦN I. TRẮC NGHIỆM PHẢN BIỆN

Câu 1

Một sinh viên cho rằng:

"Chỉ cần có động lực mạnh thì chắc chắn sẽ học giỏi."

Theo quan điểm duy vật biện chứng, nhận định nào phù hợp nhất?

A. Đúng vì ý thức quyết định hoàn toàn kết quả học tập.

Theo quan điểm duy vật biện chứng, nhận định nào phù hợp nhất?

- A. Đúng vì ý thức quyết định hoàn toàn kết quả học tập.
- B. Đúng vì động lực là yếu tố duy nhất tạo nên thành công.
- C. Chưa đầy đủ vì kết quả học tập còn phụ thuộc vào điều kiện vật chất, phương pháp học và môi trường xã hội.
- D. Sai vì ý thức không có vai trò trong hoạt động học tập.

Đáp án: C

Câu 2

Một công ty liên tục tăng số lượng nhân viên nhưng năng suất không cải thiện.

Tình huống này phản ánh hạn chế nào?

- A. Chưa xuất hiện mâu thuẫn.
- B. Chỉ thay đổi về lượng nhưng chưa tạo ra biến đổi về chất.
- C. Chất luôn quyết định lượng.
- D. Mọi thay đổi về lượng đều dẫn đến thay đổi về chất ngay lập tức.

Đáp án: B

Câu 3

Một người cho rằng:

"Công nghệ AI sẽ thay thế hoàn toàn con người trong mọi lĩnh vực."

Đánh giá nào mang tính biện chứng nhất?

- A. Hoàn toàn đúng.



B. Hoàn toàn sai.

C. Đúng trong mọi điều kiện lịch sử.

D. Cần xem xét trong mối liên hệ với trình độ công nghệ, đặc điểm công việc và bối cảnh xã hội cụ thể.

Đáp án: D

Câu 4

Trong quá trình khởi nghiệp, thất bại ban đầu khiến doanh nghiệp phải thay đổi mô hình kinh doanh và sau đó phát triển mạnh hơn.

Ví dụ này phản ánh rõ nhất quy luật nào?

A. Quy luật lượng - chất.

B. Quy luật phủ định của phủ định.

C. Quy luật đồng nhất.

D. Quy luật ngẫu nhiên tuyệt đối.

Đáp án: B

Câu 5

Nếu chỉ nhìn vào điểm số cuối kỳ để đánh giá toàn bộ năng lực học tập của sinh viên thì đã mắc hạn chế nào?

A. Nhìn nhận sự vật trong mối liên hệ.

B. Đồng nhất hiện tượng với bản chất.

C. Phân tích nguyên nhân và kết quả.



D. Vận dụng tư duy lịch sử cụ thể.

Đáp án: B

Câu 6

Một người khẳng định:

“Tôi tin điều đó là đúng nên nó chắc chắn đúng.”

Theo nhận thức luận Mác - Lênin, hạn chế lớn nhất của lập luận này là gì?

- A. Chưa dựa trên thực tiễn kiểm nghiệm.
- B. Thiếu logic hình thức.
- C. Không có yếu tố cảm tính.
- D. Không có khái niệm triết học.

Đáp án: A

PHẦN II. TỰ LUẬN VẬN DỤNG TÌNH HUỐNG THỰC TẾ

Câu 1

Vận dụng quy luật lượng - chất để phân tích quá trình xây dựng kỹ năng sử dụng trí tuệ nhân tạo (AI) của sinh viên trong thời đại số.

Câu 2

Một sinh viên dành nhiều thời gian xem video học tập nhưng kết quả học tập không cải thiện đáng kể.

Hãy vận dụng quan điểm thực tiễn của triết học Mác - Lênin để phân tích nguyên nhân và đề xuất giải pháp.

Hãy vận dụng quan điểm thực tiễn của triết học Mác - Lênin để phân tích nguyên nhân và đề xuất giải pháp.

Câu 3

Sự phát triển của mạng xã hội đã làm thay đổi mạnh mẽ cách con người tiếp nhận thông tin.

Hãy vận dụng nguyên lý về mối liên hệ phổ biến để phân tích tác động của hiện tượng này đối với cá nhân và xã hội.

Câu 4

Trong nhiều doanh nghiệp hiện nay, chuyển đổi số tạo ra áp lực thay đổi lớn đối với người lao động.

Hãy vận dụng quy luật thống nhất và đấu tranh của các mặt đối lập để phân tích quá trình này.

Câu 5

Một số ngành nghề truyền thống đang bị thay thế bởi công nghệ tự động hóa.

Hãy vận dụng quy luật phủ định của phủ định để đánh giá xu hướng phát triển này.

Câu 6

Nhiều sinh viên phụ thuộc vào AI để làm bài tập và nghiên cứu.

Dựa trên mối quan hệ giữa nhận thức và thực tiễn, hãy phân tích những lợi ích, hạn chế và điều kiện để sử dụng AI hiệu quả trong học tập.

PHẦN III. RUBRIC CHẤM ĐIỂM TỰ LUẬN (THANG 10 ĐIỂM)

Áp dụng cho tất cả các câu tự luận.



PHẦN III. RUBRIC CHẤM ĐIỂM TỰ LUẬN (THANG 10 ĐIỂM)

Áp dụng cho tất cả các câu tự luận.

Tiêu chí	Mô tả	Điểm tối đa
Xác định đúng vấn đề triết học	Nhận diện chính xác quy luật, nguyên lý hoặc phạm trù cần vận dụng	2.0
Trình bày kiến thức lý luận	Giải thích đúng nội dung triết học liên quan	2.0
Vận dụng vào tình huống thực tế	Liên hệ đúng với bối cảnh được nêu, không kể lại lý thuyết đơn thuần	3.0
Phân tích phản biện	Chỉ ra mặt tích cực, hạn chế, mâu thuẫn hoặc điều kiện tác động	2.0
Diễn đạt và lập luận	Bố cục logic, lập luận chặt chẽ, thuật ngữ chính xác	1.0
Tổng		10.0

THANG ĐÁNH GIÁ NĂNG LỰC

Điểm	Mức độ
9.0 - 10.0	Xuất sắc: hiểu sâu bản chất triết học, vận dụng linh hoạt, có tư duy phản biện
8.0 - 8.9	Tốt: nắm chắc lý thuyết, liên hệ thực tế rõ ràng
6.5 - 7.9	Khá: hiểu vấn đề nhưng phân tích chưa thật sâu
5.0 - 6.4	Đạt yêu cầu: nắm kiến thức cơ bản nhưng vận dụng còn hạn chế
Dưới 5.0	Cần bổ sung: thiên về học thuộc, thiếu phân tích và liên hệ thực tiễn